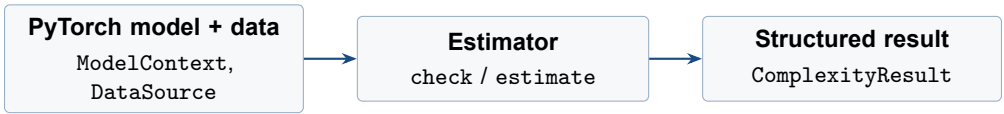
DL Complexity: one interface, different measures

Alyabushev, Dementiev, Kurdyukov, Chirkov

`dl_complexity`

Evaluate model complexity through a common PyTorch-facing API, while preserving the meaning of each measure.



Information criteria

AIC, BIC, WAIC

Fit and complexity penalties

`native_ic`

Code lengths

Two-part and prequential MDL

Description of model/data

`nats` or `bits`

Risk bounds

Compression-based bounds

Generalization guarantees
Risk scale, under assumptions

One calling convention does not imply one scale or a universal ranking.

Core objects and data flow

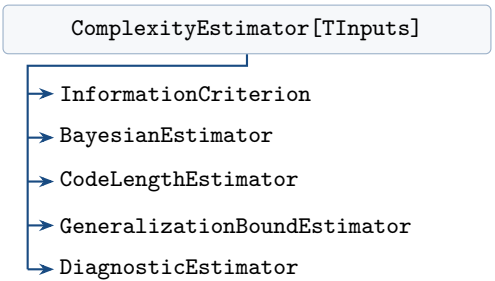


Direct preparation is method-specific; posterior samples are supplied explicitly.
Prepared records can also be passed without a model.

- A loss is not automatically a negative log-likelihood; the relation must be explicit.
- Training is explicit. A recipe stores instructions, not an implicit call to fit.

Base classes and extension points

Estimator inheritance



Branches point from a base to its subclasses.

Public protocol

`check(...)` → `RequirementReport`

`estimate(...)` → `ComplexityResult`

Pass `context` + `data`, or `inputs`.

Implementing a method

- Declare `input_type`.
- Implement `_estimate(inputs)`.
- Optional checks: `_check_inputs`.
- For a direct route, add `_check_context` and `_prepare_context`.

Separate interfaces, not estimator subclasses

`ModelTransform.apply(model)` `ModelCodec.encode(model)` / `decode(artifact)`

User workflow: check, estimate, reuse

Direct route: model + observations

```
x = torch.ones(4, 1, dtype=torch.float64)
y = torch.tensor([1., 2., 4., 5.],
                 dtype=torch.float64)[:, None]
model = nn.Linear(1, 1, bias=False,
                  dtype=torch.float64)
with torch.no_grad():
    model.weight.fill_(3.)  # analytic MLE
context = ModelContext(model,
    likelihood=GaussianLikelihood(sigma=1.))
data = DataSource.from_tensors(x, y)
bic = BIC()
report = bic.check(context, data=data)
if report.ready:
    result = bic.estimate(context, data=data)
```

Imports and the runnable example are in `presentation/README.md`.

Prepared route

```
stats = FitStatistics(
    log_likelihood=ll,
    k=1, n=4)
prepared = bic.estimate(
    inputs=stats)
```

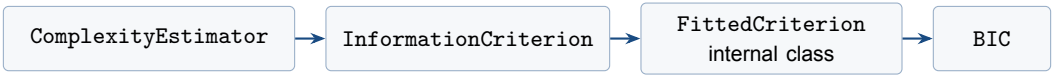
`ll`: total natural log-likelihood at the fitted parameters. Reuse the same statistics for AIC/BIC/HQIC without another model pass.

`ComplexitySuite`

Runs named estimators and returns an `EvaluationOutcome` per method. No implicit fitting, shared cache or cross-measure ranking.

Fit explicitly; `check` inspects readiness; `estimate` computes the measure.

BIC inside the architecture



Method-specific calculation

$$\text{BIC} = \underbrace{-2 \log L(\hat{\theta})}_{\text{deviance}} + \underbrace{k \log n}_{\text{penalty}}$$

```
ll, k, n = self._statistics(inputs)
deviance = -2 * ll
penalty = ll.new_tensor(k * math.log(n))
```

Exact excerpt: `criteria/bic.py`, `BIC._estimate`.

Shared preparation

`FittedCriterion` obtains `FitStatistics`: total log-likelihood, parameter count and sample count.

Standard output

`BIC` returns `bic`, `deviance`, `penalty` in `quantities`.

Unit: `native_ic`; normalization: `total`.

Formula cost: $O(1)$ after preparation; the direct route also evaluates the model. BIC assumes a regular finite-dimensional MLE setting.

`check().ready` **does not establish MLE optimality or BIC's asymptotic assumptions.**

Proof of concept: analytic reference vs library

$y = (1, 2, 4, 5), \quad y_i \sim \mathcal{N}(\mu, 1), \quad \hat{\mu} = \bar{y} = 3, \quad n = 4, \quad k = 1.$

$$\sum_i (y_i - \hat{\mu})^2 = 10, \qquad \ell(\hat{\mu}) = -\frac{4}{2}\log(2\pi) - \frac{10}{2}.$$

Quantity	Analytic reference	Model + data	FitStatistics
Deviance	17.35150826563738	17.35150826563738	17.35150826563738
Penalty	1.38629436111989	1.38629436111989	1.38629436111989
BIC	**18.73780262675727**	**18.73780262675727**	**18.73780262675727**

Standard result: `native_ic`, `total`

Both public routes agree with the independent formula to 10^{-12}.

- Five known-variance Gaussian cases pass the selected analytical tests.
- This is an integration PoC, not a full benchmark or a proof for arbitrary neural networks.

Source: `tests/test_analytical_criteria.py`, `test_gaussian_mean_mle_known_variance`.

BIC uses shared preparation and returns the standard structured result.